AccessRefinery: Fast Mining Concise Access Control Intents on Public Cloud

by Ning Kang, Peng Zhang and Jianyuan Zhang at ANTS lab.

Java 17

tests passing

paper FSE2026

About AccessRefinery

AccessRefinery automatically mines access control intents from IAM (Identity and Access Management) policies. These intents help users verify policy correctness. Compared with AWS Access Analyzer, AccessRefinery accelerates mining by ~10-100x and reduces the number of intents by up to ~10x.

To accelerate intent mining, **AccessRefinery** uses our Multi-Theory Constraint Preprocessor (**MCP**) to speed up multi-round SMT solving by preprocessing constraints into bit-vector constraints.
For intent reduction, **AccessRefinery** computes a compact set that covers all mined intents by solving a minimum set-cover problem.
Moreover, we design **MCP** as a module separate from **AccessRefinery**, allowing other researchers to reuse **MCP** flexibly.

For technical details, see our FSE 2026 paper: AccessRefinery: Fast Mining Concise Access Control Intents on Public Cloud.

Structure

Since AWS Access Analyzer is not open source and provides only a Command-Line Interface (CLI), we also reimplemented Access Analyzer for evaluation. We distinguish the two versions as

the **Reimplemented Access Analyzer** and the **CLI-based Access Analyzer**.

- `data/`:
  - `Correctness/`: Dataset for correctness experiments.
  - `Scalability_05Keys/`: Synthetic dataset for scalability experiments.
  - `Scalability_06Keys/`: Synthetic dataset for scalability experiments.
- `accessrefinery/`: Implementation of AccessRefinery.
  - `bdd/`: Implementation of the binary decision diagram backend used by MCP.
  - `mcp/`: Implementation of the Multi-Theory Constraint Preprocessor (MCP).
  - `refinery/`: Implementation of intent mining and reduction.
- `baselines/`:
  - `accessanalyzer-reimpl`: Reimplementation of Access Analyzer.
  - `accessanalyzer-cli`: Scripts for invoking AWS Access Analyzer via CLI.
- `pom.xml`: Maven root configuration.
- `tools/`: Scripts for running the experiments.
- `docs/`: Stores documents such as auto-generated JavaDoc.
  - `mcp-javadoc`
  - `accessrefinery-javadoc`
- `paper_figures/`: Scripts for plotting the figures in the paper.
- `archive_results/`: Archived experimental results.

Setup

Prerequisites

See Requirements and Installation for detailed setup instructions (Linux, Java, Z3, and jq).

Build

From the project root directory, run:

```
mvn clean package
```

The build generates the following JAR packages in `target/`:

- `mcp-1.0.jar` for **MCP**, which can be reused in other projects for fast multi-round SMT solving.
- `accessrefinery-1.0.jar` for **AccessRefinery**.
- `accessanalyzer-1.0.jar` for the reimplemented **Access Analyzer**.

For development in VS Code IDEA, see details.

Using Multi-Theory Constraint Preprocessor (MCP)

MCP is a data structure for fast multi-round SMT solving. It supports regular expressions, IP prefixes/bit-vectors, ranges, and sets.

Reuse in Another Project

Install `target/mcp-1.0.jar` into your local Maven repository:

```
mvn install:install-file \
    -Dfile=target/mcp-1.0.jar \
    -DgroupId=org.ants \
    -DartifactId=accessrefinery \
    -Dversion=1.0 \
    -Dpackaging=jar \
    -DgeneratePom=true
```

Then add the dependency to your `pom.xml`:

```
<dependencies>
    <dependency>
        <groupId>org.ants</groupId>
        <artifactId>accessrefinery</artifactId>
        <version>1.0</version>
    </dependency>
</dependencies>
```

Example

This example follows the example in the paper (line 414). The example is also included in MCPFactoryTest.java and runs automatically during `mvn package`.

```
mvn test
```

Suppose we have the following IAM policy and a target intent, `Intent_6` (`Resource`: `dept*/user1.txt`, `IpAddress`: `112.0.0.0/24`).

```
{
    "Statement": [
        {
            "Effect": "Allow",
            "Resource": ["dept*/user1.txt", "dept1/user*.txt"],
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp": ["112.0.0.0/24", "113.0.0.0/24"]
                }
            }
        },
        {
            "Effect": "Deny",
            "NotResource": "dept1/user*.txt",
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp": "112.0.0.0/24"
                }
            }
        },
        {
            "Effect": "Deny",
            "NotResource": "dept*/user1.txt",
            "Condition": {
                "IpAddress": {
                    "aws:SourceIp" : "113.0.0.0/24"
                }
            }
        }
    ]
}
```

To check the satisfiability of three formulas, $\neg I_6 \wedge P \neg I6 \wedge P$, $I_6 \wedge \neg P I6 \wedge \neg P$, and $I_6 \wedge P I6 \wedge P$, we use the following code based on **MCP**.

```java
package com.example;
import org.batfish.datamodel.Prefix;
import org.junit.Assert;
import org.mcp.core.MCPBitVector;
import org.mcp.core.MCPFactory;
import org.mcp.core.MCPFactory.MCPType;
import org.mcp.variables.statics.LabelType;

public class Main {
    public static void main(String[] args) {
        MCPFactory mcp = new MCPFactory(MCPType.BDD);
        mcp.addVar("Res", LabelType.REGEXP, "dept*/user1.txt");
        mcp.addVar("Res", LabelType.REGEXP, "dept1/user*.txt");
        mcp.addVar("IP", LabelType.PREFIX, Prefix.parse("112.0.0.0/24"));
        mcp.addVar("IP", LabelType.PREFIX, Prefix.parse("113.0.0.0/24"));
        mcp.updates();

        MCPBitVector res1 = mcp.getVar("Res", "dept*/user1.txt");
        MCPBitVector res2 = mcp.getVar("Res", "dept1/user*.txt");
        MCPBitVector ip1 = mcp.getVar("IP", Prefix.parse("112.0.0.0/24"));
        MCPBitVector ip2 = mcp.getVar("IP", Prefix.parse("113.0.0.0/24"));
        MCPBitVector s1 = (res1.or(res2)).and(ip1.or(ip2));
        MCPBitVector s2 = res1.not().and(ip1);
        MCPBitVector s3 = res2.not().and(ip2);
        MCPBitVector policy = s1.diff(s2).diff(s3);
        MCPBitVector intent6 = res1.and(ip1);

        // ¬I6∧P is satisfiable.
        Assert.assertTrue(!policy.and(intent6.not()).isZero());
        // I6∧¬P is unsatisfiable.
        Assert.assertTrue(policy.not().and(intent6).isZero());
        // I6∧P is satisfiable.
        Assert.assertTrue(!policy.and(intent6).isZero());
    }
}
```

Using AccessRefinery

AccessRefinery builds on **MCP** for IAM intent mining and reduction. In this repository, **MCP** is already integrated into **AccessRefinery**, so you can use it directly without a separate installation.

To run **AccessRefinery**, use:

```
java -jar target/accessrefinery-1.0.jar [options]
```

Command-line options:

- `-h, --help` : Show help information.
- `-m, --mine` : Enable intent mining.
- `-r, --reduce` : Enable intent reduction.
- `-f, --file <path>` : Input path for policy files (must be under `data/`).
- `-s, --sat` : Use SAT to encode bit-vectors (default is BDD).
- `--round <number>` : Number of mining rounds (to reduce experimental bias).

Example:

```
java -jar target/accessrefinery-1.0.jar -m -r --round 1 -f data/Correctness
```

The command produces logs similar to the following:

```
[INFO] 2026-04-05 22:51:33 : ----------[ AccessRefinery Mode ]-------------
[INFO] 2026-04-05 22:51:33 : input  path: data/Correctness
[INFO] 2026-04-05 22:51:33 : output path: result/Correctness
[INFO] 2026-04-05 22:51:33 : ----------< 1th policy - 11_allow_allow_equal.json >---------
[INFO] 2026-04-05 22:51:33 : [1/6]  finish parser policy
[INFO] 2026-04-05 22:51:33 : [2/6]  finish ECs calculation
...
```

Results are generated in the `results/Correctness/` directory and include:

- `xxx.json` : The intents for each policy.
- `xxx.csv` : Statistics for multi-round SMT solving for each policy.
- `summary.txt` : Summary statistics for all policies in a folder.

In addition, one file is generated in the current path:

- `accessrefinery.log` : Records the running log.

Evaluation Reproduction

This section describes how to reproduce the paper's conclusions. All archived results are available in `archive_results/`.

> We omit the results for the real-world datasets because of commercial restrictions.

We recommend skipping reproduction of results from the **reimplemented Access Analyzer** (because it takes a very long time) and the **CLI-based Access Analyzer** (because the setup is complex and requires AWS account registration, billing setup, and CLI credential configuration). We still provide instructions for developers.

- See details for the **Reimplemented Access Analyzer**.
- See details for the **CLI-based Access Analyzer**.

Reproducing AccessRefinery Archived Results

The following scripts invoke `target/accessrefinery-1.0.jar` and reproduce the **AccessRefinery** results.

```
# The execution takes about 5min
sh tools/accessrefinery/running_bdd_miner.sh

# The execution takes about 5min
sh tools/accessrefinery/running_sat_miner.sh

# The execution takes about 10min
sh tools/accessrefinery/running_bdd_reducer.sh

# The execution takes about 1h
sh tools/accessrefinery/running_sat_reducer.sh
```

The following directories are generated under `results/`:

- `accessrefinery_bdd_miner_10rs/` : Intent mining results for 10 rounds with JavaBDD.
- `accessrefinery_sat_miner_10rs/` : Intent mining results for 10 rounds with MiniSAT.
- `accessrefinery_bdd_reducer_10rs/` : Intent mining and reduction results for 10 rounds with JavaBDD.
- `accessrefinery_sat_reducer_3rs/` : Intent mining and reduction results for 3 rounds with MiniSAT.

> Because the MiniSAT-based reduction experiment runs very slowly, we report only three rounds for that experiment.

Mapping results to paper conclusions

This section presents two representative conclusions. See details for the full explanation.

1. **AccessRefinery** is 10-100x faster than **Access Analyzer** for intent mining.

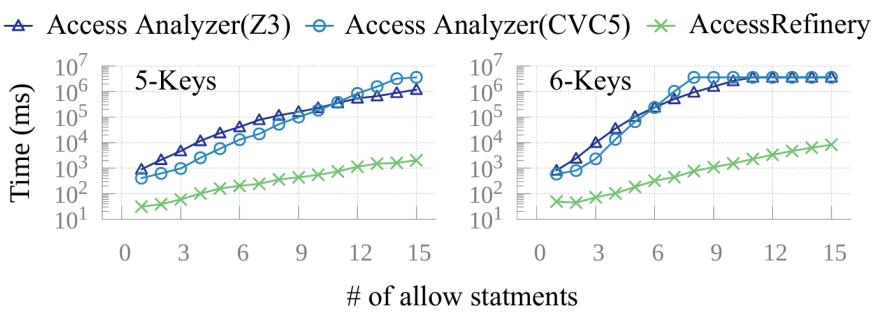

Fig. 12. The total time of mining intents for synthetic datasets.

Required logs:

- `accessrefinery_bdd_miner_10rs/`
  - `Scalability_05Keys/summary.txt`
  - `Scalability_06Keys/summary.txt`

`TotalTimeAverage` denotes the average runtime over 10 rounds for **AccessRefinery**.

- `accessanalyzer_z3_unreduced/`
  - `Scalability_05Keys/summary.csv`
  - `Scalability_06Keys/summary.csv`
- `accessanalyzer_cvc5_unreduced/`
  - `Scalability_05Keys/summary.csv`

- `Scalability_06Keys/summary.csv`

`Total Time (s)` denotes the runtime for the **reimplemented Access Analyzer**. We report one round because it is slow.

2. **AccessRefinery** can reduce the number of intents by up to 10x after reduction.

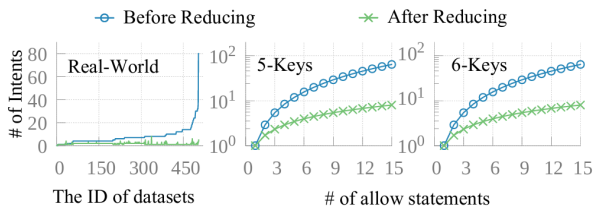

Fig. 11. The number of intents generated before *reducing intents* and after *reducing intents*. Note that the x-axis for the synthetic datasets represents the number of allowed statements.

Required logs:

- `accessrefinery_bdd_reducer_10rs/`
  - `Scalability_05Keys/summary.txt`
  - `Scalability_06Keys/summary.txt`

`NumberMCI` denotes the number of intents before reduction, and `NumberRRI` denotes the number of intents after reduction.

Plotting the Figures

We also provide scripts for drawing the figures in the paper. The following commands install `gnuplot` and generate all figures used in the experiments.

```
sudo apt install gnuplot
cd paper_figures
sh draw.sh
```

The generated figures are saved in `paper_figures/results/`.

Contact

- Ning Kang (kangning2018@qq.com)
- Peng Zhang (p-zhang@xjtu.edu.cn)

License

Apache-2.0 License, see LICENSE.